

The one question that decides everything

If I shuffled these items, would the list still make sense?

Yes → `<ul>`

Favorite foods, hobbies, shopping list, friends. Order is cosmetic.

No → `<ol>`

Recipe steps, directions, class schedule, top-10 list. Order carries meaning.

Two kinds of list

Unordered — `<ul>`

```
<ul>
  <li>Pizza</li>
  <li>Burgers</li>
  <li>Tacos</li>
</ul>
```

- Pizza
- Burgers
- Tacos

browser draws the bullets

Ordered — `<ol>`

```
<ol>
  <li>Wake up</li>
  <li>Eat breakfast</li>
  <li>Catch the bus</li>
</ol>
```

1. Wake up
2. Eat breakfast
3. Catch the bus

browser draws the numbers

Two tags always — container and item

Container

`<ul>` or `<ol>`. Declares “a list starts here.”

Item

`<li>` — list item. One entry in the list. Used inside either container.

The container decides bullets vs numbers. The `<li>` items never change.

The rule that catches everyone

Only `<li>` goes directly inside `<ul>` or `<ol>`. Put whatever you want *inside* the `<li>` — text, links, bold. Just not next to it.

```
<ul>
  <p>Pizza</p> ❌
</ul>

<ul>
  <li><strong>Pizza</strong></li> ✅
</ul>
```

Why use a real list and not dashes?

You could type `- Pizza` in a paragraph and it would *look* the same. So why bother?

A screen reader announces a real list:

Same principle as `<strong>` over `<b>`. Meaning is the point. Appearance can be changed later; meaning

"List, 3 items. Item 1 of 3: Pizza."

A dash is just a character. A list is structure.

cannot.

Nested lists — the one rule

The inner list goes **inside** the `<li>` — before its closing tag.

Correct

```
<ul>
  <li>Breakfast
    <ul>
      <li>Cereal</li>
      <li>Toast</li>
    </ul>
  </li> ← closes AFTER inner list
  <li>Lunch
    <ul>
      <li>Sandwich</li>
    </ul>
  </li>
</ul>
```

Wrong — the trap

```
<li>Breakfast</li> ← closed too early
<ul>
  <li>Cereal</li>
</ul>
```

This often renders fine. It breaks the moment you try to style or indent the sub-list — because the inner `<ul>` is now a *sibling* of the outer `<li>`, not a child.

Trace the nesting out loud

```
<ul>           ① outer list opens
  <li>Breakfast  ② Breakfast item opens
    <ul>         ③ inner list opens
      <li>Cereal</li> ④ inner item
    </ul>        ⑤ inner list closes
  </li>         ⑥ Breakfast item closes
  <li>Lunch      ⑦ next item begins
</ul>          ⑧ outer list closes
```

The `</li>` at step ⑥ has to wait for the inner list to fully close at step ⑤. That is the whole rule.

Lists of links — how every nav menu is built

```
<ul>
  <li><a href="index.html">Home</a></li>
  <li><a href="about.html">About</a></li>
  <li><a href="#contact">Contact</a></li>
</ul>
```

Yesterday's lesson, inside today's. Three L4 rules visible in one code block:

- Relative `href` for your own pages
- `#` in the `href` for same-page jumps
- Link text that stands alone out of context

The `<a>` goes **inside** the `<li>`. The item is the container, the link is its content.

Emmet shortcuts for lists

TYPE THIS → TAB

GET THIS

<code>ul>li*3</code>	a <code></code> with three empty <code></code> items
<code>ol>li*5</code>	an <code></code> with five items
<code>ul>li*2>ul>li*3</code>	a two-item list, each item holding a three-item sub-list

Reading the shorthand: `>` means *inside*. `*` means *repeat*. Chain them to build a whole list in one line.

Emmet only helps if you can write the HTML yourself. On the test there is no Tab key. If you can't nest a list by hand, you can't fix one Emmet built wrong.

Common mistakes to avoid

MISTAKE	WHAT GOES WRONG
Nested list outside the <code></code>	Closing <code></code> before the inner <code></code> starts. Renders fine at first, breaks when styled.
<code><p></code> inside <code></code>	Only <code></code> goes directly inside a list. Anything else is a guess the browser will make differently from how you meant.
<code></code> for things with no order	Numbering your hobbies. If shuffling wouldn't break it, it's a <code></code> .
Forgetting <code></code>	Fine on a flat list. Breaks the moment you nest — the browser guesses where the item ends and guesses wrong.
Mixing <code></code> and <code></code>	<code></code> is the item tag, not the container. You always need both. <code></code> alone is not a list.

Today's syntax — quick reference

GOAL	CODE
Bullet list	<code>Item</code>
Numbered list	<code>Step</code>
One item	<code>Text</code>
Nested list	inner <code></code> before the closing <code></code>
Link inside a list	<code>Text</code>
Emmet — three items	<code>ul>li*3</code> + Tab
Emmet — nested	<code>ul>li*2>ul>li*3</code> + Tab

What is next. Your pages can now point anywhere and organize anything. Lesson 6 adds images — the tag that comes with two required attributes and the accessibility rule that students skip most often.

Keep this sheet. Every navigation menu on every website you visit from today on is a `<ul>` full of `<li><a>` — and now you can see it.